

Polyglot AI Team OS : 主 Agent 不单独接 LLM

HTML 版交付：先做成可读的网页文档，再从 HTML 导出 PDF

这版先保留文档原意：主 Agent 的问题不是缺一个专属大模型，而是缺 workspace introspection。正确路线是先补确定性的工作区感知层，再让主系统变得不瞎。

Polyglot AI Team OS : 主 Agent 不单独接 LLM 的路线说明

1. 这次方向修正的核心结论

这次最重要的结论不是“再给主 Agent 接一个 Claude”，而是：

****主 Agent 不应该单独做一层 LLM。****

原因很简单：你现在遇到的问题，并不是主 Agent 缺少抽象推理能力，而是它连最基础的工作区感知都不完整。

所以真正该补的不是“再加一个脑子”，而是：

- 让主 Agent 能看见当前 workspace
- 让主 Agent 能用确定性的方式理解目录结构
- 让主 Agent 能基于当前项目状态做更靠谱的 plan / reply

这比额外套一层 LLM 更符合现在这个阶段，也更符合 Polyglot AI Team OS 的产品哲学：

- low-token
- structured coordination
- deterministic runtime
- 主系统不装懂
- 能用代码做确定性事情的地方，就不要先交给模型

2. 当前问题到底是什么

你现在主 Agent 的问题，不是“太蠢”，而是它本质上更像：

- 规则路由器
- 状态回复器
- 命令分发器

而不是：

- 真正能看懂当前 repo 的 planner

它现在大概能做这些事：

- 根据用户一句话估计复杂度
- 选择一个 worker
- 返回 status / board / report / history
- 把 `/run` 交给 monitor 和 worker 去跑

但它不太能做这些事：

- 看看当前目录结构
- 说清楚这个 repo 现在长什么样
- 判断核心入口和关键模块
- 回答“我现在先该看哪里”
- 把项目上下文带进 planning

所以根因非常明确：

****主 Agent 缺的不是 LLM，而是 workspace introspection。****

3. 为什么不建议单独给主 Agent 再套一层 Claude

直觉上，好像“不会看目录，那就让 Claude 帮它看”很合理。

但如果直接这么做，会引入几个问题：

3.1 会把简单问题复杂化

你现在要解决的只是：

- 看当前目录
- 看关键文件
- 看当前 session 产物
- 看项目骨架

这些事情本地代码完全能稳定完成。

如果为了这件事单独接一个 LLM，就变成了：

- 多一个推理层
- 多一套 prompt
- 多一层状态传递
- 多一类不稳定行为

3.2 会提高 token 成本

目录理解如果完全交给 LLM，最容易出现的问题就是：

- 把很多无关文件也带进上下文
- 每次都重新消耗 token
- repo 稍大就开始贵

这和你本来要坚持的 low-token coordination 是冲突的。

3.3 会降低可控性

目录扫描这类事，本来最适合确定性逻辑：

- 文件在不在
- 目录是什么
- 入口文件是什么
- session 路径是什么

这些东西本地代码做，输出是稳定的；
交给模型做，输出就会漂。

3.4 会让主 Agent 再次变成“看起来聪明，但不够扎实”

你现在最需要的是：

- 可解释
- 可验证
- 可复用
- 可持续增强

而不是：

- 再包一层模型后“感觉聪明了”

4. 正确的解决思路：给主 Agent 补一个工作区感知层

所以正确路线不是：

- “主 Agent 接一个专属 LLM”

而是：

- “主 Agent 接一个本地、确定性的 workspace introspection layer”

这层应该做的事情非常明确：

4.1 扫描当前 workspace

至少输出这些内容：

- workspace 根目录
- 顶层目录和文件
- 核心入口文件
- 关键模块目录
- session artifacts 目录
- 语言 / 项目类型的粗略判断

4.2 产出一个结构化 snapshot

例如：

```
```json
{
 "workspace_dir": "/home/ubuntu/polyglot-ai-team",
 "top_level_entries": [
 "polyglot_cli.py",
 "polyglot_ai/",
 "skills/",
 "artifacts/",
 "feishu_bridge.py",
 "feishu_listener.py",
 "README.md"
],
 "important_files": [
 "polyglot_cli.py",
 "polyglot_ai/main_agent.py",
 "polyglot_ai/runtime.py",
 "polyglot_ai/v1_worker.py"
],
 "important_dirs": [
 "polyglot_ai/",
 "skills/",
 "artifacts/"
],
 "session_dir": "artifacts/sessions/default/",
 "repo_kind_guess": "python terminal-first AI team runtime"
}
```

---

### ### 4.3 再产生一个文本 summary

例如：

```
```text
```

Workspace Summary

- root: /home/ubuntu/polyglot-ai-team
- top level: polyglot_cli.py, polyglot_ai/, skills/, artifacts/, feishu_bridge.py, feishu_listener.py, README.md
- key entrypoints: polyglot_cli.py, polyglot_ai/main_agent.py, polyglot_ai/runtime.py
- key runtime modules: polyglot_ai/v1_worker.py, polyglot_ai/worker_adapters.py
- session artifacts: artifacts/sessions/default/
- repo kind: python terminal-first AI team runtime

```
```
```

这就已经足够让主 Agent“不瞎”。

---

## ## 5. 这件事要怎么接进当前代码骨架

现在这个方案最适合直接接进现有 repo，而不是推翻重来。

### ### 5.1 Runtime 负责真实感知

最合适的落点是：

- `polyglot\_ai/runtime.py`

因为 Runtime 已经掌握：

- workspace\_dir
- session\_dir
- artifacts\_dir
- 事件和状态文件位置

所以让 Runtime 负责工作区感知是顺手的，也符合职责。

建议新增：

- `workspace\_snapshot()`
- `workspace\_summary\_text()`

### ### 5.2 MainAgent 负责消费 summary

在：

- `polyglot\_ai/main\_agent.py`

里做两件事：

1. 扩展 `chat\_reply()`

- 当用户问：
  - 当前目录
  - workspace
  - repo
  - 项目结构
  - 有什么文件
- 返回 `workspace\_summary\_text()`

2. 扩展 `plan\_goal()` / `build\_team\_plan()`

- 在生成 Team Plan 前，读一份最小 workspace summary
- 让 plan 不再只依赖用户一句目标

### 5.3 CLI 可以加一个显式命令

可以新增：

- `/workspace`  
- `/repo`

这样主 Agent 的目录感知能力就不只是隐式问答，也可以显式调用。

---

## 6. 第一阶段到底要明确做什么

如果接下来只看“明确任务”，那第一阶段应该非常具体：

### Task 1：新增工作区快照能力

目标：让 Runtime 能稳定产出当前 workspace 的结构化快照。

建议内容：

- 顶层文件/目录
- 关键文件存在性
- 关键目录存在性
- 当前 session artifacts 路径
- repo kind guess

### ### Task 2 : 新增工作区文本摘要能力

目标 : 把 snapshot 转成主 Agent 和用户都能直接消费的 summary。

### ### Task 3 : 让主 Agent 能回答目录问题

目标 : 用户问“当前目录有什么”“这个项目长什么样”时，输出 summary，而不是只给 release 文件列表。

### ### Task 4 : 让 Team Plan 带一点项目上下文

目标 : planning 时能结合当前 repo，而不是只看用户的一句话。

---

## ## 7. 这一阶段不该做什么

为了避免再次失控，这一阶段也必须明确“不做什么”：

- 不单独给主 Agent 接一层专属 LLM
- 不让主 Agent 自己全仓自由浏览
- 不把所有文件都塞进 planning 上下文
- 不先做复杂的语义级 repo 理解
- 不为了“聪明”而牺牲稳定性和可控性

这一阶段要解决的是：

**\*\*让主 Agent 有基础视野，不是让主 Agent 变成黑盒超脑。\*\***

---

## ## 8. 完成后的效果应该是什么

这一刀做完之后，主 Agent 至少应该表现成这样：

### 用户问：当前目录有什么？

它能答：

- 这是一个 Python 的 terminal-first AI team runtime 项目
- 顶层有 `polyglot\_cli.py`、`polyglot\_ai/`、`skills/`、`artifacts/` 等
- 核心模块是 `main\_agent.py`、`runtime.py`、`v1\_worker.py`
- 当前 session 产物在 `artifacts/sessions/<session\_id>/`

### 用户问：我现在该看哪里？

它能答：

- 如果你要看入口，从 `polyglot\_cli.py` 开始
- 如果你要看主控制层，看 `polyglot\_ai/main\_agent.py`
- 如果你要看状态和 session，看 `polyglot\_ai/runtime.py`
- 如果你要看执行闭环，看 `polyglot\_ai/v1\_worker.py`

### 用户问：这个项目现在像什么？

它能答：

- 它现在更像一个已经有 runtime 雏形的本地 Team OS 原型
- 当前强项在状态、控制信号、自愈闭环、host integration 雏形
- 当前短板在主 Agent 的 workspace perception 和 Worker Fabric 的 profile 化

这就够了。

---

## ## 9. 最终结论

这次的方向修正非常重要，因为它明确了一件事：

**\*\*主 Agent 不需要先变成一个新的 LLM 系统，它需要先变成一个不瞎的系统。\*\***

所以现在最正确的做法是：

1. 不单独给主 Agent 接 LLM
2. 用本地代码补一层 workspace introspection
3. 让 MainAgent 消费这层结构化摘要
4. 先把目录理解、项目概览、planning context 这三件事做好

这条路比“再套一个 Claude”更简单、更稳、更便宜，也更符合 Polyglot AI Team OS 本来的方向。

---

## ## 10. 一句话版本

**\*\*目录扫描自己做，目录理解先做成确定性 summary；主 Agent 先获得视野，再谈更复杂的智能。\*\***